Automated PPA and Hardware Benchmarking

To enable systematic design-space exploration, a set of Python scripts that automate the entire flow from configuration generation to hardware performance measurement were developed. The scripts are located in `picosoc/` and are designed to be run from the top-level directory.

Overview of the automation flow

1. **`interactive_ppa.py`** – interactively selects binary CPU parameters (e.g., `BARREL_SHIFTER`, `ENABLE_MUL`, `ENABLE_FAST_MUL`), cache parameters (`CACHE_LINES`, `CACHE_WORDS_PER_LINE`), and cache module. Generates `configs.csv` with one row per configuration.
2. **`run_ppa_fixed.py`** – reads `configs.csv`; for each configuration, copies Verilog sources, overrides parameters, runs Yosys synthesis, nextpnr place-and-route, icetime timing analysis, and icepack to produce `design.bin`. It also runs functional tests using `icebreaker_tb.v` (SoC simulation) and, if they pass, compiles the benchmark firmware (`group5_benchmark.c`) into `benchmark.bin`.
3. **`run_hardware_bench.py`** – for a given configuration, programs the FPGA with `design.bin` and `benchmark.bin`, captures the UART output, and parses `CYCLES=0x…` and `INSTNS=0x…` to compute CPI. Results are appended to `hardware_results.csv`.
4. **`run_all.sh`** – orchestrates the whole process: asks to regenerate `configs.csv`, then loops over each configuration, calling `run_ppa_fixed.py` and `run_hardware_bench.py` sequentially. It saves results incrementally, so a partial run does not lose already collected data.

Required environment

- **OSS CAD Suite** (yosys, nextpnr, icepack, icetime, iceprog) – source the environment before running.
- **RISC-V toolchain** (`riscv32-unknown-elf-gcc`, etc.) – must be in `PATH`.
- **Python 3** with `pyserial` installed.
- The iCEBreaker board attached and accessible via USB (typically `/dev/ttyUSB1`).

Usage

```
cd ~/final   # or the top-level directory containing the `picosoc/` folder
./run_all.sh
```

PPA automation script variations: progression from working_configs to final

The five uploaded versions of flow in **`historical_ppa_flow`** show the automation flow moving from a basic software-only PPA sweep into a controlled build, test, benchmark and hardware-measurement pipeline. Across the versions, the core idea remains constant: generate PicoRV32/iCEBreaker configurations, build each configuration in an isolated directory, collect area

and timing metrics, and use the outputs to compare design trade-offs. The progression is incremental rather than a full rewrite. Each version aimed at improving a specific part of the flow: path handling, parameter generation, test coverage, CSV consistency, per-configuration reruns, benchmark firmware generation, cache compatibility and final hardware CPI capture over serial. However, progressive versions often broke or lost functionality in other areas.

All analysis results should be interpreted as representative rather than definitive. The observed correlations, trends, relative rankings and orders of magnitude are expected to be broadly reliable; however, exact numerical values may vary if the experiments are repeated. This is due to the large number of configuration settings, tool outputs, generated files and measurement conditions involved, some of which were difficult to track and control perfectly throughout the project.

Version	**Key Functional Change**	**Effect on Flow**	**Errors / Oversights**
`working_configs`	Baseline automation flow for generating configs, patching Verilog, running tests, and collecting PPA data. `ENABLE_FAST_MUL` was forced to zero in the PPA runner.	Established the basic automated sweep: generate `configs.csv`, copy sources, patch parameters, run tests, run Yosys / nextpnr / icetime, and write `ppa_config_fixed.csv`.	`ENABLE_FAST_MUL` was forced to zero because configurations failed on resource usage. The hardcoded override was later forgotten until trying to replicate exact CPI, making it a significant oversight.
`main_tues`	Improved project-root and `picosoc/` path handling. Added stronger testbench parameter propagation and a more detailed analysis script.	Made the flow more robust and better aligned with the real project structure, especially the location of `picorv32.v`, tests, Makefiles, and `picosoc/` files.	The missing comma between `ENABLE_IRQ` and `TWO_STAGE_SHIFT` remained, creating a combined parameter such as `enable_irqtwo_stage_shift` and causing CSV / analysis confusion.
`team_final`	Fixed the missing comma bug, added `s` as a skip/use-default option, removed the duplicated proceed prompt, and added divider and I-cache tests.	Made configuration generation cleaner and improved functional checking before synthesis. Failed RV32IM tests could stop synthesis unless overridden.	The generated test data became richer, but the analysis script mostly remained the simpler original version, so not all new test information was fully used.
`friday_clean_start`	Added `ENABLE_FAST_MUL` as a selectable parameter and removed the earlier forced-zero override.	Allowed fast multiply to be genuinely explored as part of the configuration sweep.	Reintroduced the possibility of configurations failing because fast multiply could exceed iCE40 UP5K resource limits.

Version	Key Functional Change	Effect on Flow	Errors / Oversights
`final`	Added pure config generation, `ACTUAL_FREQ_MHZ`, fixed CSV schemas, `--config` runs, `icepack`, benchmark compilation, `icache_wrapper.v`, and hardware serial capture.	Completed the flow from configuration generation through PPA, simulation, bitstream generation, firmware compilation, hardware programming, and CPI capture.	Most complete version, but still depends on hardware setup, serial-device detection, benchmark firmware generation, and careful handling of the remaining fast-multiply override.
